



CSE3433 SOFTWARE ARCHITECTURE

PROGRAMME: COMPUTER SCIENCE (SOFTWARE ENGINEERING)

SEMESTER II 2025/2026

GROUP 6

PROJECT TITLE:

SMARTLEARN: CLOUD-NATIVE LEARNING MANAGEMENT SYSTEM (LMS)

LECTURER: DR. MOHAMAD NOR HASAN

NO.	GROUP MEMBERS	MATRIC NO.
1.	PUTERI IREENE HAZWANI BINTI MUHAMAD HAIZAR	S75702
2.	NURAIN NAZIRAH BINTI ABDULLAH	S75415
3.	NURUL ALESSA BINTI MOHD NIZAM	S76278

TABLE OF CONTENT

1.0 Introduction.....	4
2.0 Problem Description.....	5
3.0 System Requirements.....	6
3.1 Functional Requirements.....	6
3.1.1 User Management Module.....	6
3.1.2 Course Management Module.....	6
3.1.3 Assignment Management Module.....	6
3.1.4 Grading Management Module.....	6
3.2 Non-functional Requirements.....	7
3.2.1 Performance.....	7
3.2.2 Availability.....	7
3.2.3 Scalability.....	7
3.2.4 Security.....	7
3.2.5 Usability.....	7
4.0 Architecture Design.....	8
4.1 Presentation Layer.....	8
4.2 Application Layer.....	8
4.3 Data Layer.....	9
4.4 Cloud Deployment Architecture.....	9
5.0 Architecture Diagrams.....	10
5.1 Context Diagram.....	10
5.2 Component/Service Architecture Diagram.....	11
5.3 Interaction Diagram.....	13
5.3.1 User Management Module.....	13
5.3.2 Course Management Module.....	14
5.3.3 Assignment Management Module.....	15
5.3.4 Grading Module.....	16
5.4 Deployment Diagram.....	17
6.0 Technology Stack.....	19
7.0 Prototype Implementation.....	20
7.1 User Management Module.....	20
7.2 Course Management Module.....	20
7.3 Assignment Management Module.....	20
7.4 Grading Management Module.....	20
8.0 Architecture Evaluation.....	21
8.1 Scalability.....	21
8.2 Maintainability.....	21
8.3 Security.....	21
8.4 Availability.....	21
8.5 Performance.....	21

9.0 Team Contributions.....	22
10.0 Conclusion and Reflection.....	24

1.0 Introduction

With educational institutions moving more of their classes online, having a reliable Learning Management System (LMS) is incredibly important. SMARTLEARN is a modern online learning portal designed to make everyday academic tasks smooth and simple. The system gives students an easy way to register, log in, enter course codes to join classes, upload assignments, and check their marks. At the same time, it allows lecturers to easily set up new courses, post assignment details with deadlines, and grade student submissions.

Instead of bundling everything into a single old-school application, SMARTLEARN separates the frontend user interface from the backend logic engine. The user interface runs on Vercel, which ensures the website loads quickly for users anywhere. The backend is built using Node.js and Express and runs on Render, which directly handles the system logic and connects securely to our cloud database. This report provides a straightforward look at how the system is put together, how data moves between its layers, and how it handles high user traffic cleanly.

2.0 Problem Description

In many educational institutions, existing Learning Management Systems (LMS) have several limitations, especially when a large number of users access the system at the same time. This often causes slow performance, delays in loading pages, and a poor user experience for both students and lecturers.

In addition, some LMS are not scalable and lack flexibility when adding new features or handling increasing numbers of users. This makes it difficult for the system to support growing academic needs.

Furthermore, many traditional LMS are built using monolithic architecture, where all components are combined into a single system. This makes the system harder to maintain, update, and improve. Any small change may affect the whole system and can take a long time to complete. As a result, system updates may interrupt ongoing academic activities and reduce overall efficiency. Therefore, there is a need to develop a modern LMS that uses cloud-native principles and layered architecture to improve system performance, scalability, flexibility, and user experience.

3.0 System Requirements

3.1 Functional Requirements

3.1.1 User Management Module

FR-1.1: The system shall allow new users to sign up by entering their name, email, password, role, and student matric number.

FR-1.2: The system shall verify user roles to ensure students cannot access lecturer features and vice versa.

FR-1.3: The system must intercept all unauthorized database requests using custom security middleware to protect user endpoints.

3.1.2 Course Management Module

FR-2.1: Lecturers shall be able to create new courses by entering a title, a short description, and a unique course code (such as CSF3433).

FR-2.2: Students shall be able to instantly join a course by entering the correct course code given by their lecturer.

FR-2.3: The system must automatically display all joined courses as cards directly on the user's personal dashboard.

3.1.3 Assignment Management Module

FR-3.1: The lecturer shall set up assignment submission links with specific instruction and submission deadlines.

FR-3.2: The students must be able to upload their assignment in digital files.

3.1.4 Grading Management Module

FR-4.1: The system must show real-time status labels for assignment such as “Not Graded”, “Draft” or “Submitted”

FR-4.2: The system shall update the students’ grade view when the lecturer key-in their grades.

3.2 Non-functional Requirements

3.2.1 Performance

The webpages hosted on Vercel must load and render completely within less than 2 seconds under normal conditions.

3.2.2 Availability

The system must maintain a 99.9% uptime rate so that students can access learning materials and upload assignments 24/7.

3.2.3 Scalability

The Node.js backend on Render must handle spikes of over 1,000 concurrent users smoothly during major submission deadlines without crashing.

3.2.4 Security

User sessions and JSON Web Tokens (JWT) must be verified through backend middleware to keep passwords and grades safe from tampering.

3.2.5 Usability

The interface must be fully responsive so that users can easily navigate dashboards and submit work on desktops, tablets, or smartphones.

4.0 Architecture Design

SMARTLEARN adopts a cloud-native 3-tier architecture consisting of the Presentation Layer, Application Layer, and Data Layer. This architectural design improves system scalability, maintainability, and security by separating the system into independent layers.

4.1 Presentation Layer

The Presentation Layer serves as the user interface of the SMARTLEARN system. It consists of a Web Application (Single Page Application) and a Mobile Application. The web application is implemented in the current prototype, while the mobile application is planned for future enhancement to provide greater accessibility across devices.

Students can register, log in, join courses, submit assignments, and view grades through this layer. Lecturers can create courses, manage assignments, and enter student grades. User requests are sent to the Application Layer through REST API communication.

4.2 Application Layer

The Application Layer acts as the core processing layer of the system and contains all business logic. It consists of several services, including User Service, Course Service, Assignment Service, and Grading Service.

The User Service manages registration, login, and role verification. The Course Service handles course creation and enrollment. The Assignment Service manages assignment creation and submission processes, while the Grading Service processes grading activities and grade updates.

In addition, the Security Component performs authentication and authorization using JSON Web Tokens (JWT) and middleware mechanisms. The Notification Component is responsible for generating notifications and communicating with external email services.

4.3 Data Layer

The Data Layer stores and manages all persistent data within the system. It consists of the User Database, Course Database, Assignment Database, and Grade Database.

Each service in the Application Layer accesses its respective database to store and retrieve information. This separation improves data organization, reduces dependency between modules, and supports future system expansion.

4.4 Cloud Deployment Architecture

SMARTLEARN is deployed using cloud-based platforms. The frontend application is hosted on Vercel, while the backend services are deployed on Render. Email notifications are delivered through AWS Simple Email Service (SES).

This cloud-native deployment approach provides better scalability, availability, and performance while allowing the system to support a large number of concurrent users.

5.0 Architecture Diagrams

5.1 Context Diagram

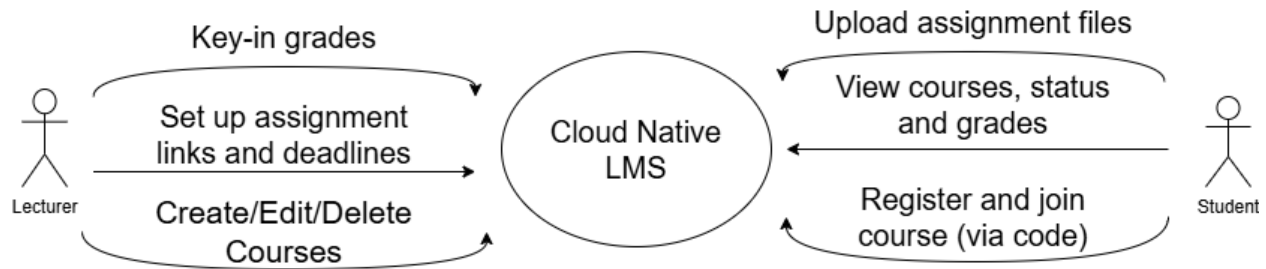


Diagram 1.0: Context Diagram

The context diagram shows how external users interact with the SMARTLEARN system at a high level. The main users are **students** and **lecturers**, who communicate directly with the system.

Students use the system to register, log in, enroll in courses, submit assignments, and view their grades. Lecturers use the system to create and manage courses, upload assignments, and evaluate student submissions.

The SMARTLEARN system acts as the central system that processes all requests and returns the results to users. This diagram helps to clearly define the system boundary and shows how data flows between users and the system without showing internal technical details.

5.2 Component/Service Architecture Diagram

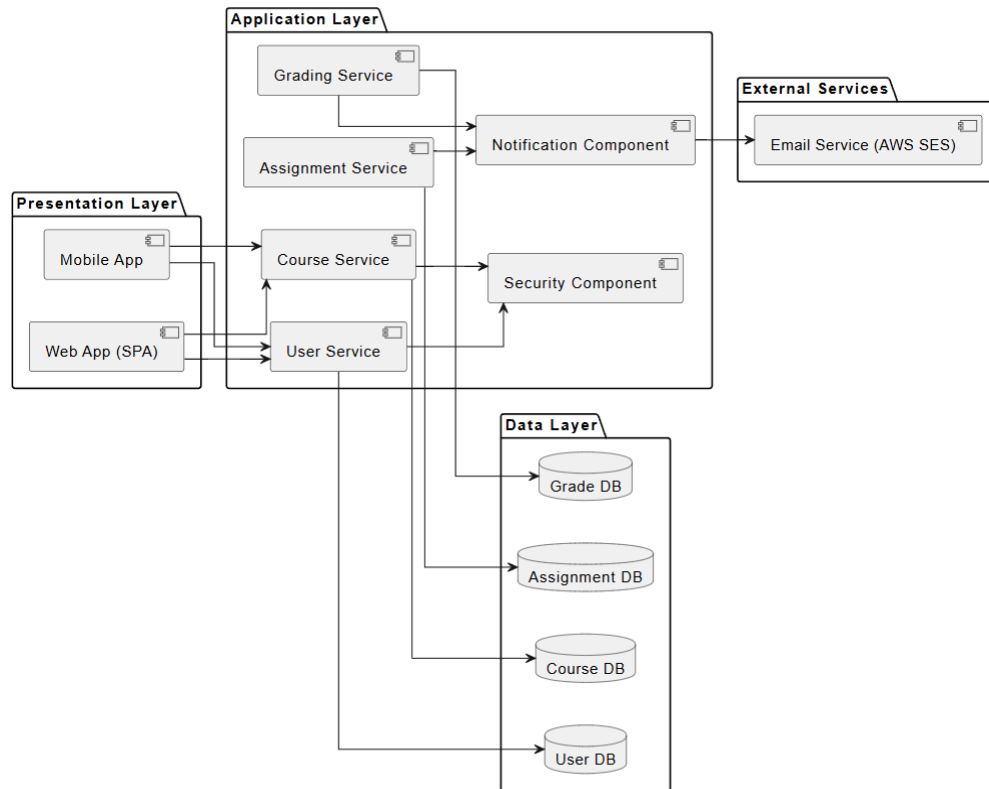


Diagram 2.0: Component/Service Diagram

The diagram illustrates the component and service architecture of the SMARTLEARN system based on a cloud-native 3-tier layered architecture. The system is divided into three main layers, namely the Presentation Layer, Application Layer, and Data Layer, with additional integration to external services. The Presentation Layer consists of the Web Application (SPA) and Mobile Application, which act as the user interface for students and lecturers to interact with the system. These applications send user requests to the Application Layer through REST API communication.

The Application Layer represents the core processing unit of the system, where all business logic is handled. It is composed of four main services, namely User Service, Course Service, Assignment Service, and Grading Service. Each service is responsible for a specific function, such as managing users, handling course operations, processing assignment submissions, and managing grading activities. In addition to these core services, the system includes supporting

components such as the Security Component and Notification Component. The Security Component is responsible for handling authentication and authorization, ensuring that only authorized users can access system features. Meanwhile, the Notification Component manages system notifications, such as sending alerts for assignment submissions and grading updates.

The Data Layer stores all system data, including the User Database, Course Database, Assignment Database, and Grade Database. Each database is used by its respective service to store and retrieve relevant information. This design helps ensure better organization of data and improves system scalability. Furthermore, the Notification Component is integrated with an external service, specifically the Email Service (AWS SES), to deliver email notifications to users.

Overall, the architecture demonstrates a well-structured and scalable system where user requests flow from the Presentation Layer to the Application Layer for processing, and then to the Data Layer for storage, before responses are returned to the user interface. This design supports modularity, maintainability, and efficient system performance in a cloud environment.

5.3 Interaction Diagram

5.3.1 User Management Module

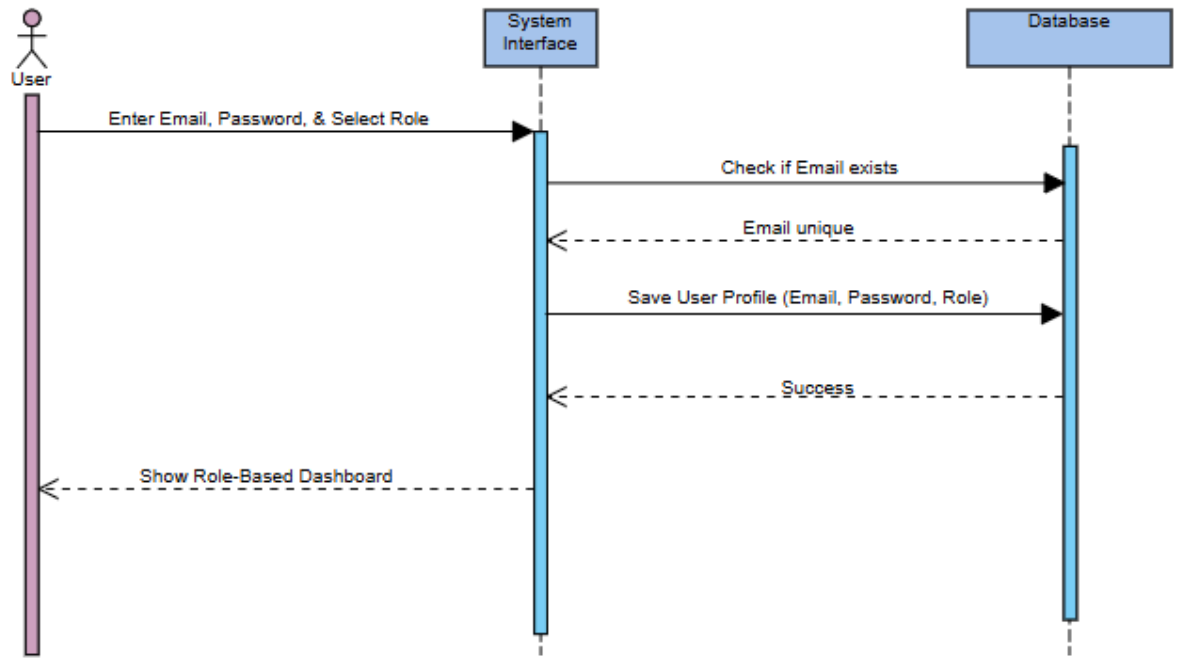


Diagram 3.1 Interaction Diagram (User Management Module)

The interaction diagram for the User Management Module illustrates a sequential process where the user first initiates the workflow by entering their email, password, and selected role into the system interface, which then triggers a validation request to the database to ensure the email address is unique. Once the database confirms the email's uniqueness, the system interface sends a secondary command to save the complete user profile into the database records, and upon receiving a successful confirmation from the database, the system interface concludes the cycle by presenting the user with a customized, role-based dashboard.

5.3.2 Course Management Module

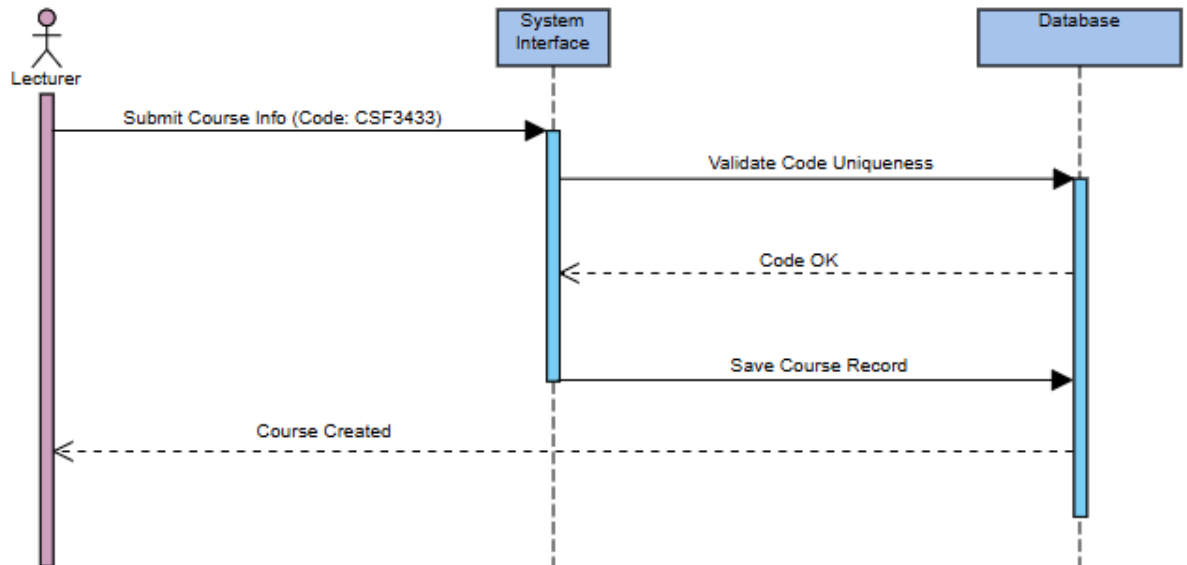


Diagram 3.2 Interaction Diagram (Course Management Module - Lecturer)

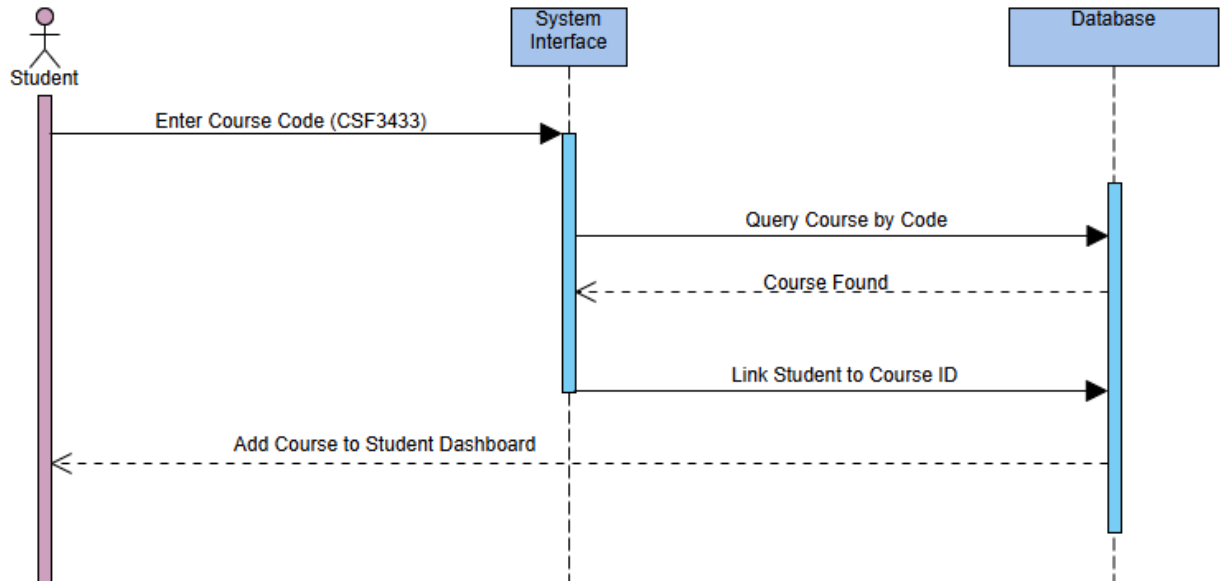


Diagram 3.2 Interaction Diagram (Course Management Module - Student)

The Course Management Module interaction diagram is divided into two main workflows. First, a lecturer initiates the creation of a new course by submitting course

information through the system interface, which then prompts the database to validate the uniqueness of the course code before saving the record and confirming to the lecturer that the course has been created.

Second, a student enters a specific course code into the interface, triggering the system to query the database for that code and, upon finding the course, link the student's ID to the course record so that the course is successfully added to the student's personal dashboard.

5.3.3 Assignment Management Module

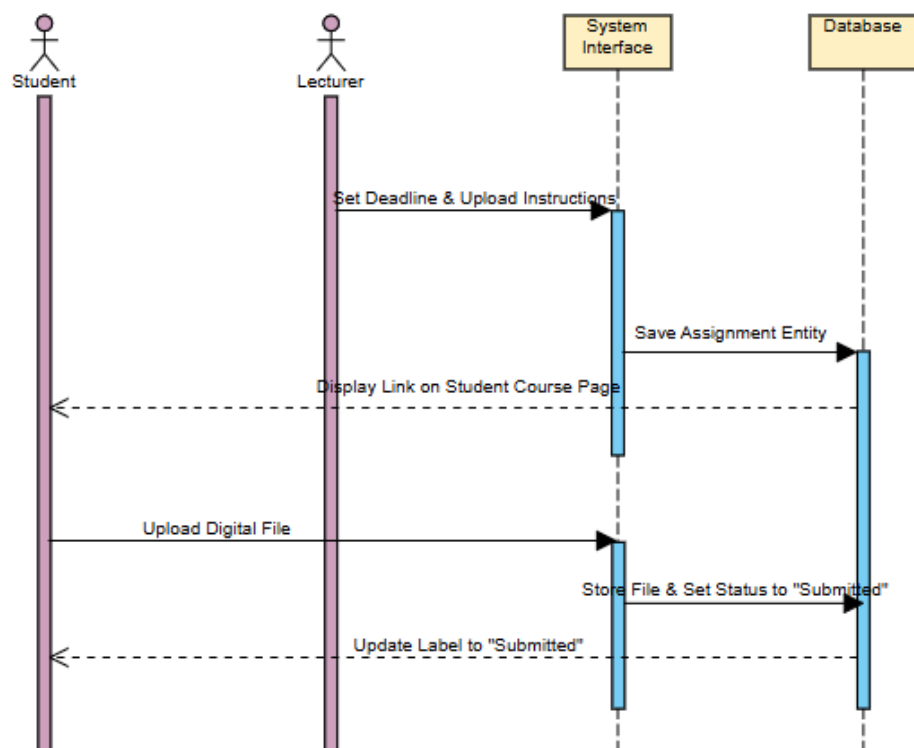


Diagram 3.3 Interaction Diagram (Assignment Management Module)

The Assignment Management Module interaction diagram depicts a two-phase process starting with a lecturer setting a deadline and uploading instructions through the system interface, which then saves the assignment entity into the database and displays a submission link on the student's course page. Subsequently, the student uploads their digital file to the system interface,

which triggers the database to store the file and update the assignment status to "Submitted" while simultaneously updating the visual label on the student's interface to reflect that the submission is complete.

5.3.4 Grading Module

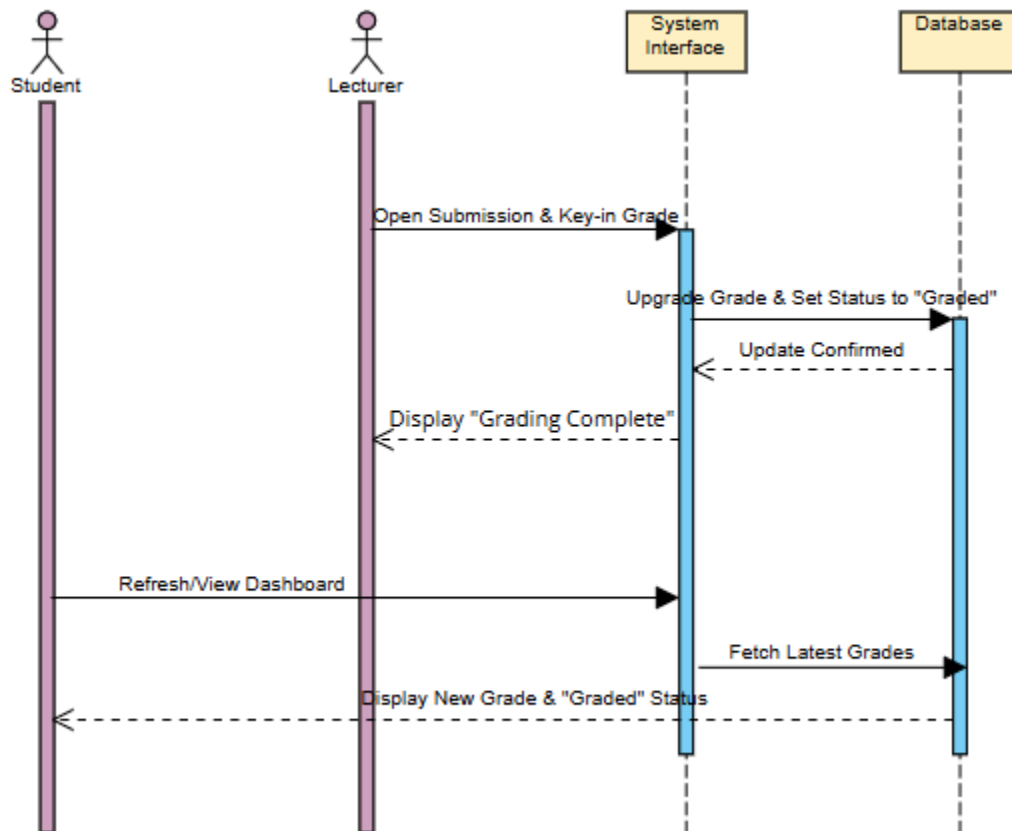


Diagram 3.4 Interaction Diagram (Grading Management Module)

The Grading Module interaction diagram shows a workflow where a lecturer first opens a student's submission and enters a grade through the system interface, which then updates the grade and sets the assignment status to "Graded" within the database. Once the database confirms the update, the lecturer receives a notification that grading is complete, and the student can subsequently refresh their dashboard to trigger the system interface to fetch the latest scores from the database and display the new grade along with the updated status.

5.4 Deployment Diagram

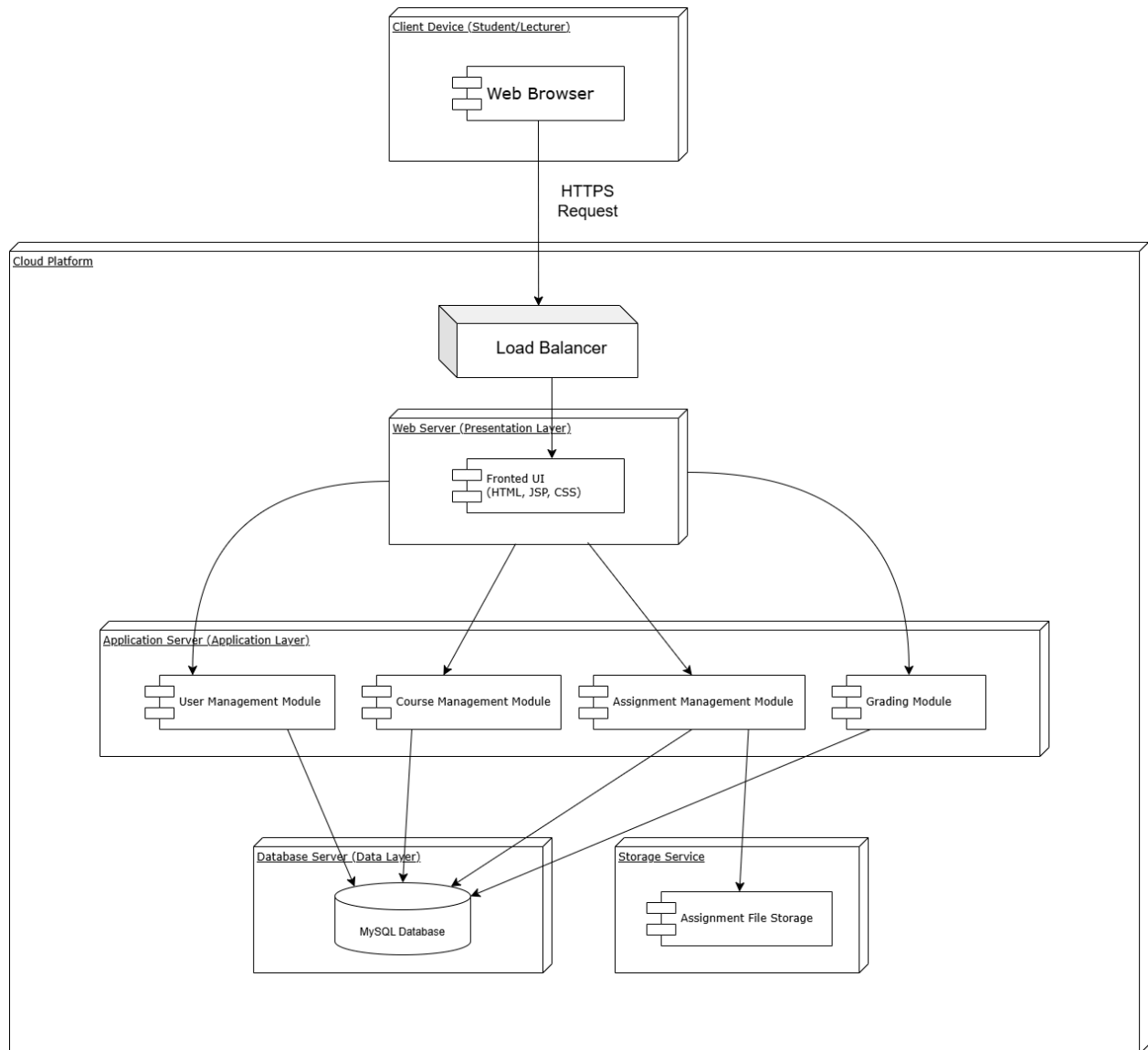


Diagram 4.0 Deployment Diagram

The deployment diagram shows how the SMARTLEARN system is physically deployed in a cloud environment. Users (students and lecturers) access the system through a web browser on their devices using an internet connection.

The system is hosted on a cloud platform and uses a **load balancer** to distribute incoming requests across servers. This helps the system handle many users at the same time without slowing down.

The system follows a **3-tier architecture**:

- The **Presentation Layer** (web server) handles the user interface
- The **Application Layer** (application server) processes system logic and modules
- The **Data Layer** (database server) stores all system data

Assignment files are stored in a separate storage service. This setup improves system performance, scalability, and availability, ensuring that the system can support many users and remain accessible at all times.

6.0 Technology Stack

Architecture Layer	Technology Used	Purpose
Presentation Layer	HTML, CSS, JavaScript	User Interface for Students and Lecturers
Presentation Hosting	Vercel	Frontend Deployment
Application Layer	Node.js, Express.js	Business Logic Processing and API Management
Security Component	JSON Web Tokens (JWT) Authentication, Middleware	Authentication & Authorization
API Communication	REST API	Communication Between Frontend and Backend
Data Layer	MongoDB	Data Storage
Cloud Backend Hosting	Render	Backend Deployment
Notification Service	AWS SES	Email Notifications
Version Control	GitHub	Source Code Management

7.0 Prototype Implementation

The SMARTLEARN prototype was developed to demonstrate the core functionalities of the proposed Learning Management System.

The prototype includes:

7.1 User Management Module

The User Management Module was developed to manage user registration, authentication, and role-based access control. This module allows students and lecturers to securely access the system according to their assigned roles.

Features implemented:

- User Registration
- User Login
- Role-Based Access Control
- Dashboard Display

7.2 Course Management Module

The Course Management Module enables lecturers to create and manage courses while allowing students to join courses using unique course codes. The module automatically displays enrolled courses on the user's dashboard.

Features implemented:

- Course Creation by Lecturers
- Course Enrollment Using Course Codes
- Course Dashboard Display

7.3 Assignment Management Module

The Assignment Management Module allows lecturers to create assignments with instructions and submission deadlines. Students can upload assignment files and monitor submission status through the system.

Features implemented:

- Assignment Creation
- Assignment Submission
- Submission Status Tracking

7.4 Grading Management Module

The Grading Management Module enables lecturers to evaluate student submissions and assign grades. Students can view updated grades and assignment status through their dashboard.

Features implemented:

- Grade Entry by Lecturers
- Grade Viewing by Students
- Real-Time Status Updates

The frontend interface was deployed using Vercel while the backend services were hosted on Render. Database integration was implemented to support data persistence and retrieval.

8.0 Architecture Evaluation

The SMARTLEARN architecture was evaluated based on several quality attributes.

8.1 Scalability

The cloud-native architecture allows the system to scale horizontally when the number of users increases. The use of cloud hosting platforms enables resources to be expanded when necessary.

8.2 Maintainability

The separation of concerns through the 3-tier architecture improves maintainability. Changes in one layer can be implemented without significantly affecting other layers.

8.3 Security

JSON Web Tokens (JWT) authentication and backend middleware ensure that only authorized users can access protected resources. Role-based authorization prevents unauthorized access to lecturer functions.

8.4 Availability

Cloud deployment on Vercel and Render provides high system availability and reliable service delivery.

8.5 Performance

The system satisfies the performance requirement by utilizing Vercel for frontend hosting and Node.js backend services, enabling webpages to load within the targeted response time of less than two seconds under normal conditions.

8.6 Usability

The SMARTLEARN interface is designed to be responsive and user-friendly, allowing students and lecturers to access the system effectively across desktops, tablets, and smartphones. The dashboard-based design improves navigation and enhances the overall user experience.

Overall, the architecture successfully meets the functional and non-functional requirements defined for SMARTLEARN.

9.0 Team Contributions

Name	Role	Core System Contributions	Architectural Execution
Puteri Ireene Hazwani binti Muhamad Haizar	Frontend Developer	• Created the fully responsive client-side interface files. • Deployed and managed the user views live on Vercel. • Hooked up asynchronous javascript fetch requests to read backend JSON API streams	Presentation Layer Design & User Experience Integration
Nurain Nazirah Binti Abdullah	Backend Developer	• Built the Node.js and Express API route files. • Programmed the custom route verification middleware and security check parameters. • Structured clean backend response	Application Layer Flow & Security Architecture

		flows.	
Nurul Alessa Binti Mohd Nizam	Full-stack Architect & Database Lead	• Designed and organized our cloud database schema tables. • Linked data drivers directly into the Express server routing pipeline. • Programmed the container scripts and managed deployment linkages on Render.	Data Layer Architecture & Cloud DevOps Orchestration

10.0 Conclusion and Reflection

SMARTLEARN was developed as a cloud-native Learning Management System designed to improve the online learning experience for both students and lecturers. Through the implementation of a 3-tier layered architecture, the system achieves better scalability, maintainability, security, and performance compared to traditional monolithic systems.

Throughout this project, the team gained valuable experience in software architecture design, cloud deployment, service-based development, and system integration. The project also improved the team's understanding of cloud-native principles and the importance of separating system responsibilities into different layers.

In conclusion, SMARTLEARN demonstrates how modern architectural practices can be applied to develop an efficient, scalable, and user-friendly learning management platform that meets the needs of contemporary educational institutions. Future enhancements may include mobile-native support, real-time notifications, and advanced learning analytics to further improve the learning experience.